

Chap.2 활성화 함수 (Activation Function)

방 수 식 교수
(bang@tukorea.ac.kr)

한국공학대학교 전자공학부

2024년도 2학기
고급머신러닝실습 & 인공지능설계실습2

[복습] Back-Propagation for DNN



■ 연쇄법칙의 이해

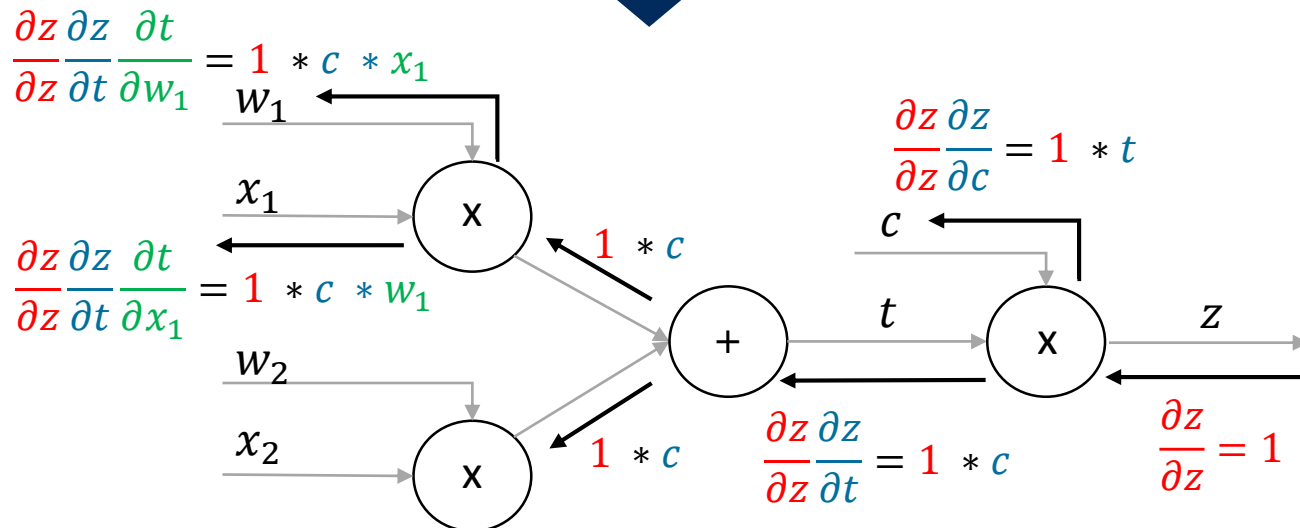
- 합성함수 : 여러 함수로 구성된 함수
- 연쇄법칙 : 합성함수의 미분은 각 함수의 미분의 곱으로 나타낼 수 있음

ex) $z = c(ax + by)$ 일 때, x 에 대한 z 의 미분값은?

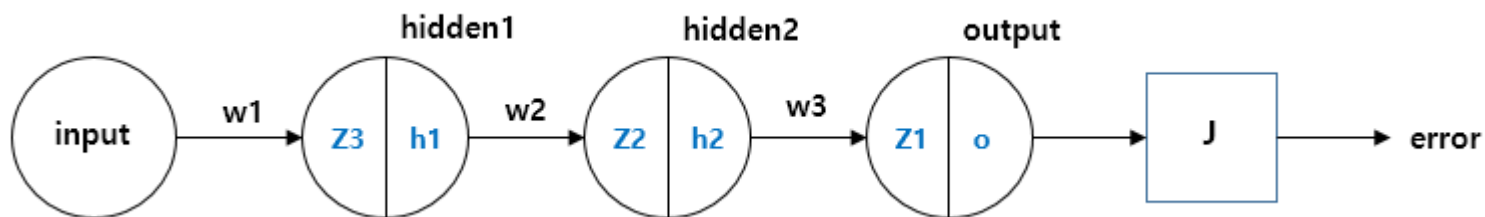
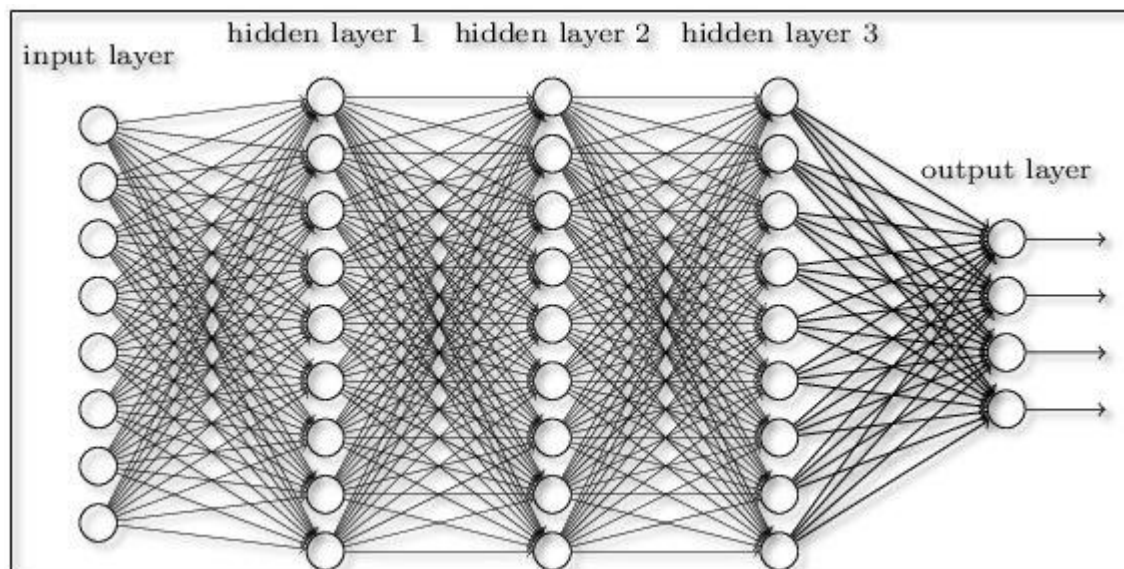
$$\begin{aligned} t &= ax + by \\ z &= ct \end{aligned} \quad , \quad \frac{\partial t}{\partial x} = a \quad , \quad \frac{\partial z}{\partial t} = c \quad , \quad \frac{\partial z}{\partial x} = \frac{\partial z}{\partial t} \frac{\partial t}{\partial x} = ac$$



그림으로 표현



Deep Learning으로의 확장



Node의 활성화 함수가 모두 동일하다고 했을 때,
 w_1 을 update하기 위해서는 활성화 함수의 도함수를 3번 곱한 값이 들어감.

■ 활성화 함수 종류

- Sigmoid 함수

- $\sigma(z) = \frac{1}{1+e^{-z}}$

- 미분이 간단 $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

- Tanh 함수

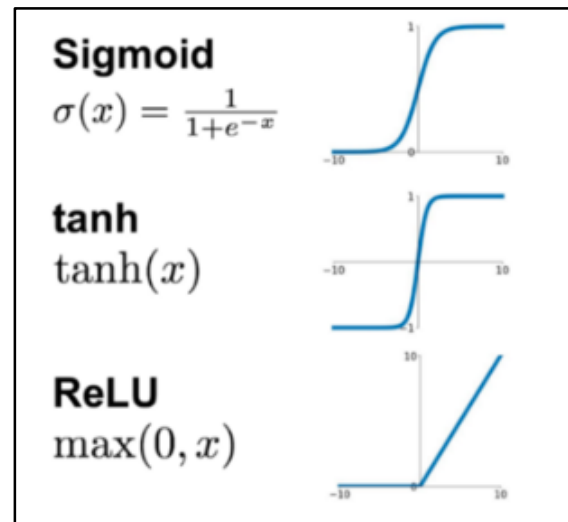
- $\tanh(z) = 2\sigma(2z) - 1$

- $\tanh'(z) = 1 - \tanh^2(z)$

- 미분가능하고 출력이 -1 ~ 1 사이

- 훈련 초기에 각 층의 출력을 원점 근처로 모으는 경향이 있음

└─> 종종 빠르게 수렴하도록 도와줌



- ReLU(Rectified Linear Unit) 함수 ──> 실제로 가장 많이 사용됨

- $ReLU(z) = \max(0, z)$ $ReLU'(z) = \begin{cases} 1 & (x \geq 0) \\ 0 & (x < 0) \end{cases}$

- Z= 0에서 미분 불가능

- 기울기가 갑자기 변해 문제 발생 가능

- 실제로는 잘 작동하기도 하며 계산 속도가 매우 빠름

Gradient Vanishing

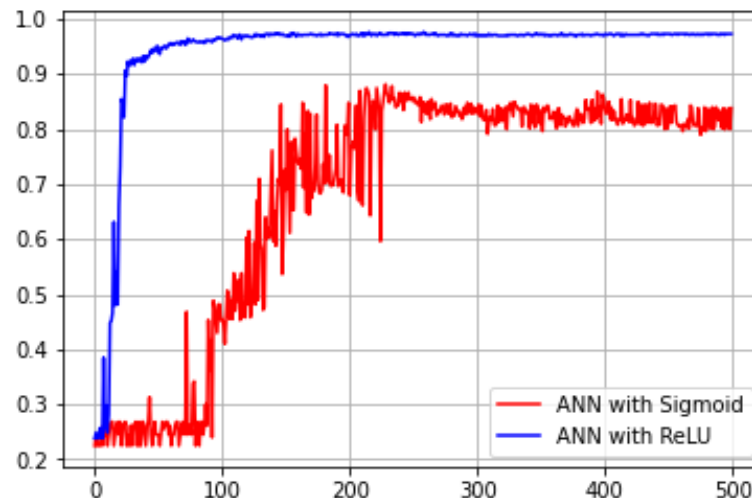


■ Sigmoid의 한계점

- 학습 시, 갈수록 기울기가 점점 작아지는 현상 발생
- 기울기가 너무 작아지면 제대로 된 학습 불가능
➡ Gradient Vanishing 문제

■ ReLU를 사용하여 해결

- 미분값이 1이기 때문에, Gradient Vanishing 문제 X
- 추가로, 계산복잡도가 낮아져 효율적인 학습

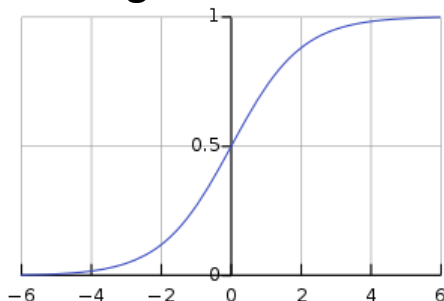


Gradient Vanishing Problem

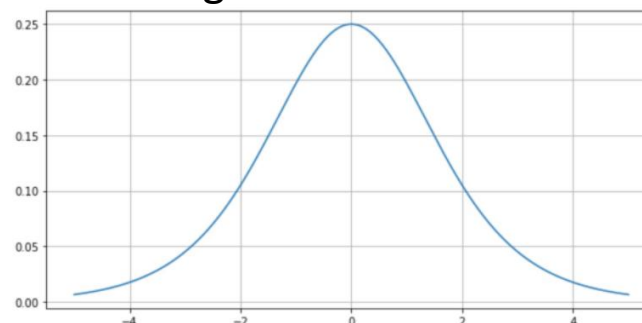


지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

Sigmoid 함수



Sigmoid의 도함수



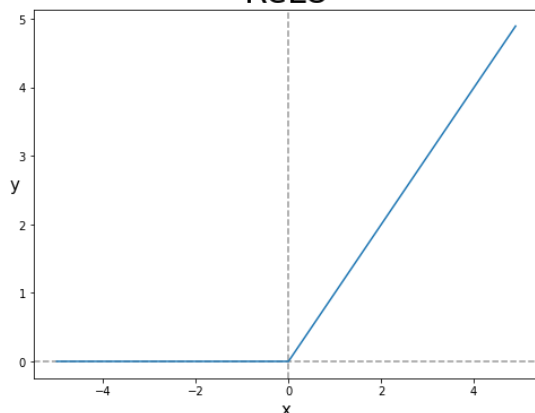
“Gradient Vanishing Problem”



도함수를 곱할수록 값이 작아져
Weight Update가 잘 이뤄지지 않음.
=> Hidden Layer가 많을수록 불리

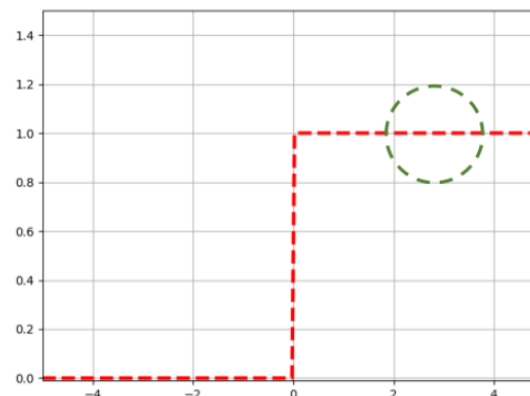
ReLU 함수

Gradient Vanishing 문제를
해결하기 위해 제안된 활성화 함수
ReLU



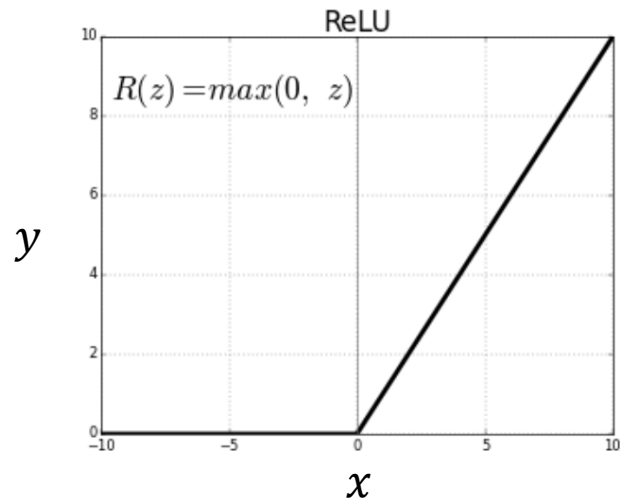
$$f(x) = \begin{cases} 0 & x < 0 \\ x & x \geq 0 \end{cases} \quad f(x) = \max(0, x)$$

dReLU(x)/dx



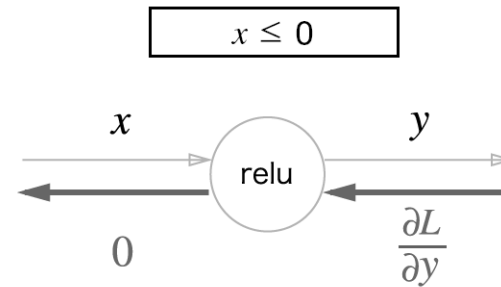
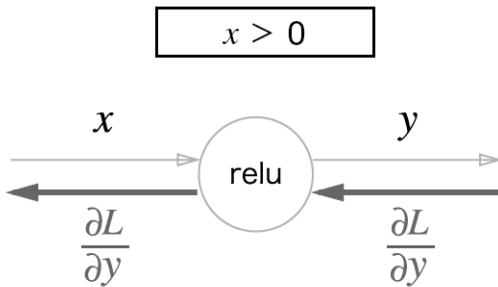
Hidden Layer가 많아도,
즉, Neural Network가 Deep해져도
Weight를 업데이트 할 수 있음

ReLU

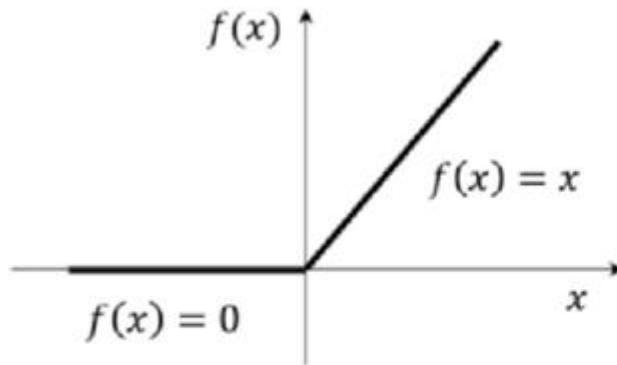


$$y = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

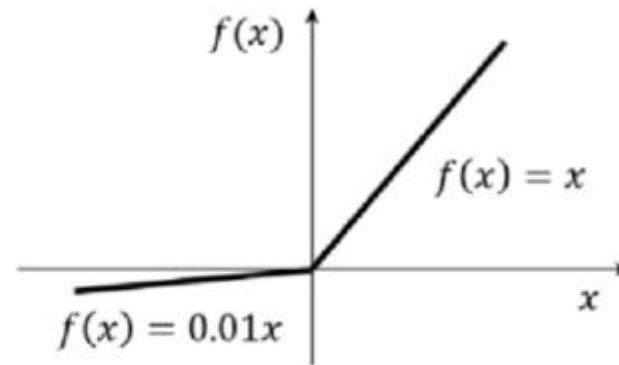
$$\frac{\partial y}{\partial x} = \begin{cases} 1 & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$



Leaky ReLU



ReLU activation function

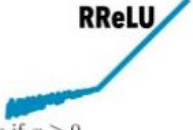
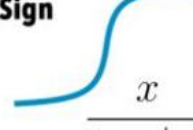
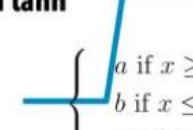
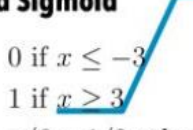
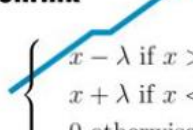
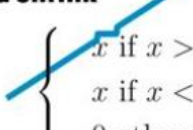


LeakyReLU activation function

미분?

다양한 Activation Function



ReLU  $\max(0, x)$	GELU  $\frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + ax^3) \right) \right)$	PReLU  $\max(0, x)$
ELU  $\begin{cases} x & \text{if } x > 0 \\ \alpha(x \exp x - 1) & \text{if } x < 0 \end{cases}$	Swish  $\frac{x}{1 + \exp -x}$	SELU  $\alpha(\max(0, x) + \min(0, \beta(\exp x - 1)))$
SoftPlus  $\frac{1}{\beta} \log(1 + \exp(\beta x))$	Mish  $x \tanh \left(\frac{1}{\beta} \log(1 + \exp(\beta x)) \right)$	RReLU  $\begin{cases} x & \text{if } x \geq 0 \\ ax & \text{if } x < 0 \text{ with } a \sim \mathcal{R}(l, u) \end{cases}$
HardSwish  $\begin{cases} 0 & \text{if } x \leq -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$	Sigmoid  $\frac{1}{1 + \exp(-x)}$	SoftSign  $\frac{x}{1 + x }$
Tanh  $\tanh(x)$	Hard tanh  $\begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$	Hard Sigmoid  $\begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x \geq 3 \\ x/6 + 1/2 & \text{otherwise} \end{cases}$
Tanh Shrink  $x - \tanh(x)$	Soft Shrink  $\begin{cases} x - \lambda & \text{if } x > \lambda \\ x + \lambda & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$	Hard Shrink  $\begin{cases} x & \text{if } x > \lambda \\ x & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$

Chap.2 실습 #1

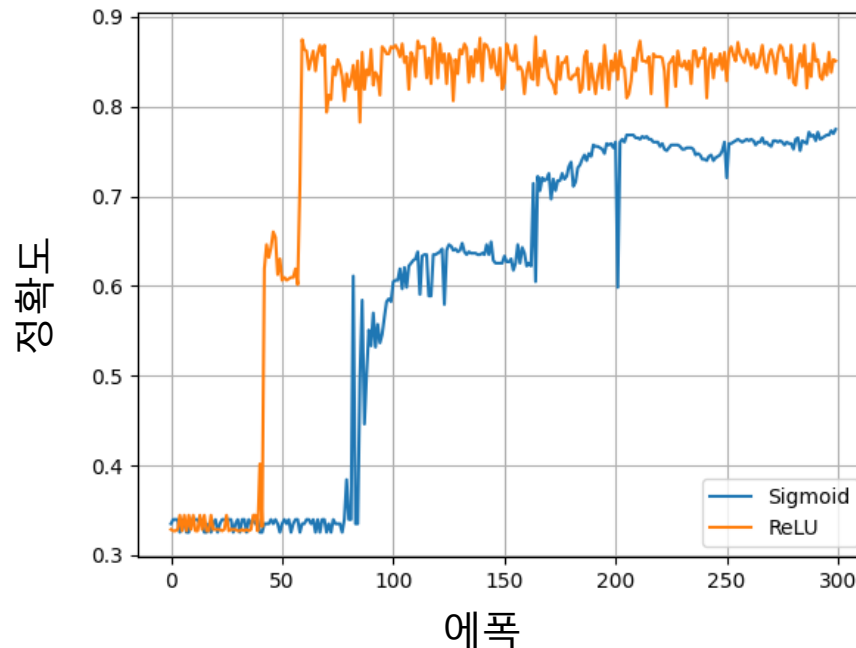


각 실습문제에 대해 *code(+ 주석), 그래프, 분석 내용 등은 필수적으로 포함시킬 것*

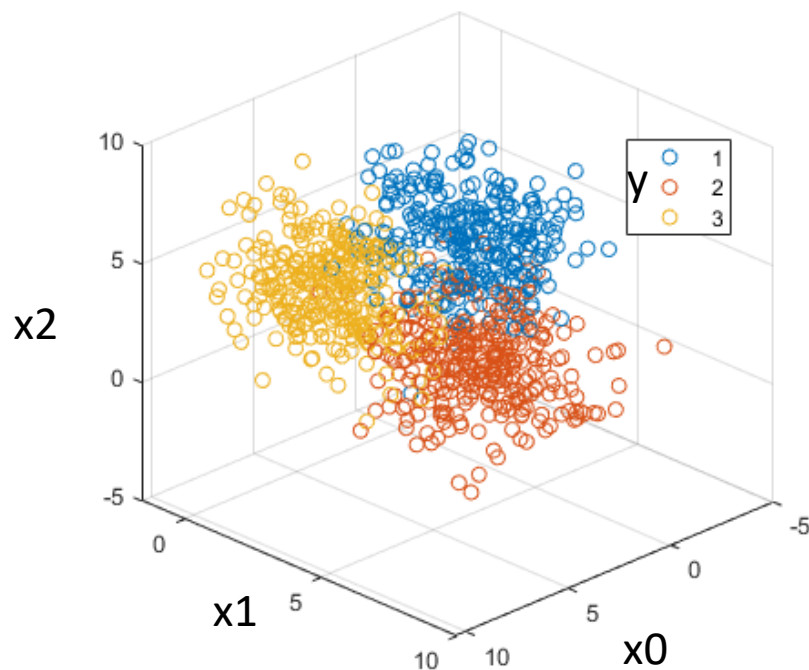
실습 #1) ReLU 사용

- Chap.1 실습 #2)에서 출력층을 제외한 모든 활성화 함수를 ReLU로 변경
- Chap.1 실습 #2)의 결과와 Chap.2 실습 #1)의 결과를 비교 분석
- 정확도 비교 이외에도 여러 분석 내용 기재
ex) 가중치 업데이트 속도를 비교하며 Gradient Vanishing을 판단 등등

예상 결과



Chap.1 & 2 실습 데이터



- 1) Class1 : $(x_0, x_1, x_2) = (2, 4, 6)$ 을 중심으로 노이즈가 추가된 데이터
- 2) Class2 : $(x_0, x_1, x_2) = (4, 6, 2)$ 을 중심으로 노이즈가 추가된 데이터
- 3) Class3 : $(x_0, x_1, x_2) = (6, 2, 4)$ 을 중심으로 노이즈가 추가된 데이터

각 Class별 300개씩 총 900개

Softmax Function

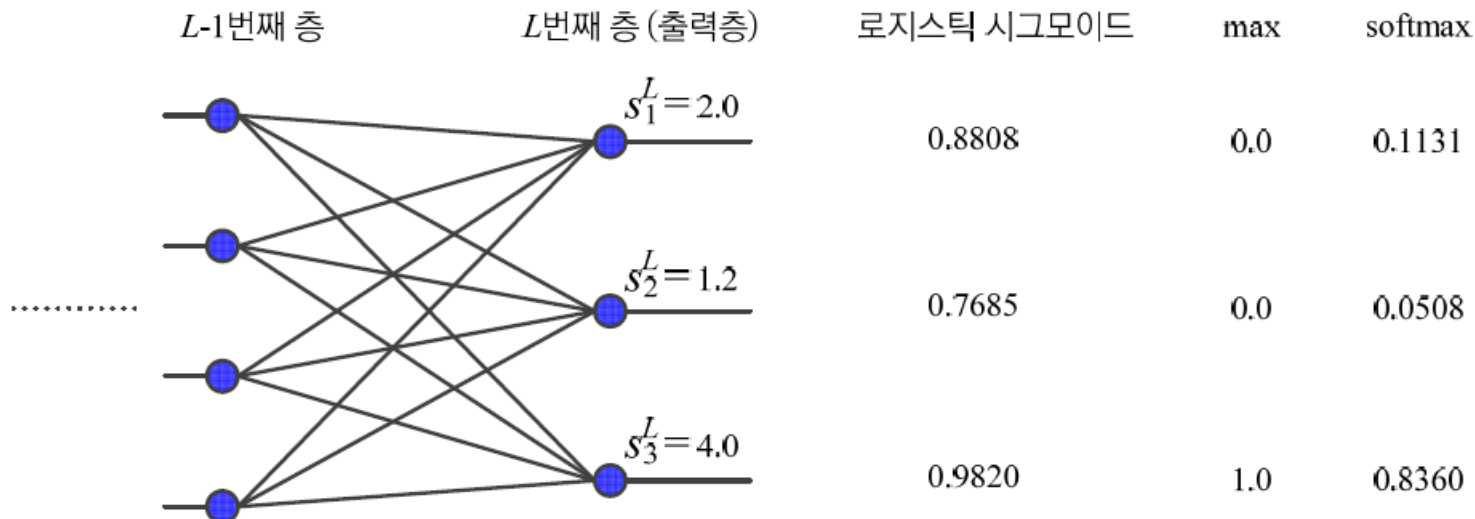
다중 분류를 위한 Softmax Function

$$\text{softmax}(z) = \frac{e^{\hat{z}_j}}{\sum_{j=0}^k e^{\hat{z}_j}} \quad (j = 0, 1, \dots, k)$$

z 는 Output node의 마지막 입력값

동작 예시

- softmax는 max를 모방(출력 노드의 중간 계산 결과 s_i^L 에서 최댓값은 더욱 활성화하고 작은 값은 억제)
- 모두 더하면 1이 되어 확률 모방



Softmax Function

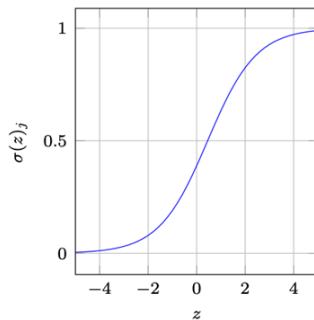


▪ Softmax 함수

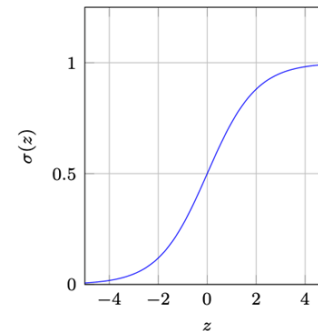
- 출력층에서 사용되는 활성화 함수
- N개의 이벤트에 대해 확률 분포를 계산하는 함수

$$\text{softmax}(z) = \frac{e^{\hat{z}_j}}{\sum_{j=0}^k e^{\hat{z}_j}} (j = 0, 1, \dots, k)$$

Softmax Function	Sigmoid Function
확률의 총 합 = 1	총 합이 1은 아님
출력층에서 사용됨	Activation 함수로 사용될 수 있음
예측한 클래스에 실제로 해당할 가능성을 나타냄 (실제 확률)	예측한 클래스에 해당할 가능성이 높지만 확률 값은 아님



(b) Softmax activation function.



(a) Sigmoid activation function.

Softmax layer

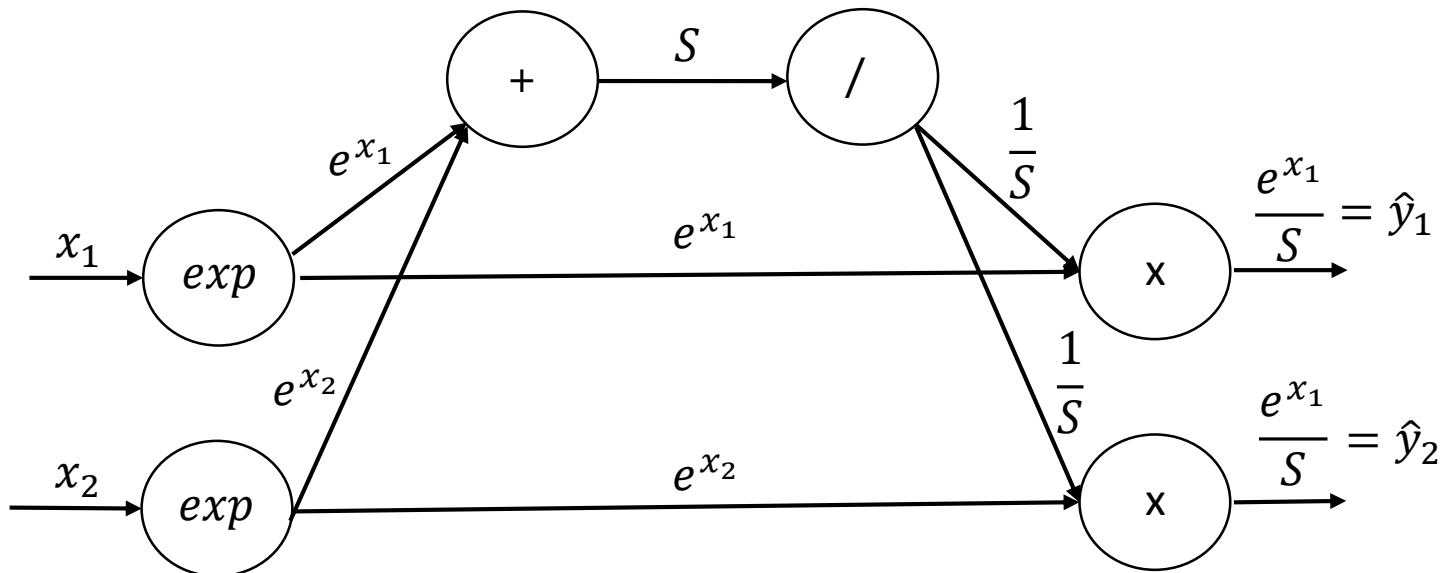


▪ Softmax 함수

- 출력층에서 사용되는 활성화 함수
- N개의 이벤트에 대해 확률 분포를 계산하는 함수

$$\text{softmax}(x) = \frac{e^{x_j}}{\sum_{j=0}^k e^{x_j}} (i = 0, 1, \dots, k)$$

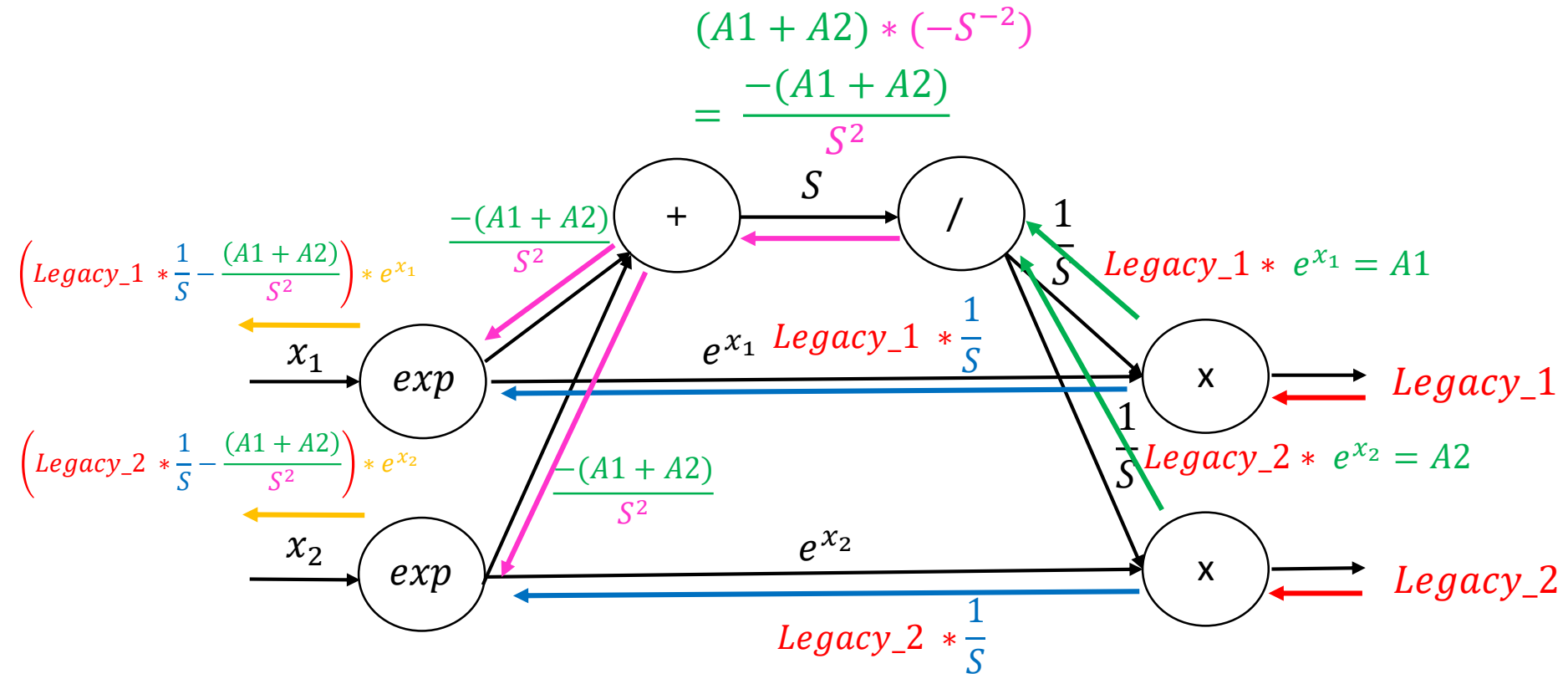
↓ forward



Softmax layer



Softmax Backward



Cross Entropy Error (CEE)

▪ Softmax를 출력함수로 사용할 때의 Cost Function

• Cross-Entropy

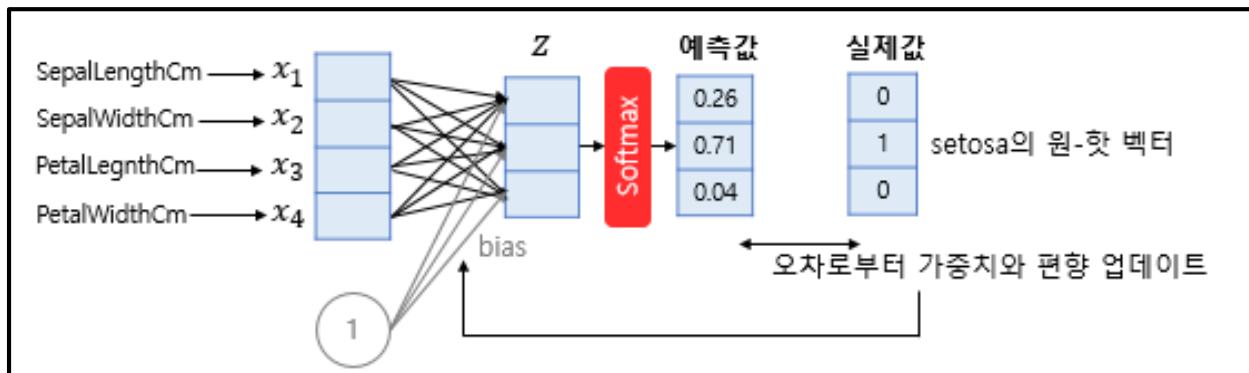
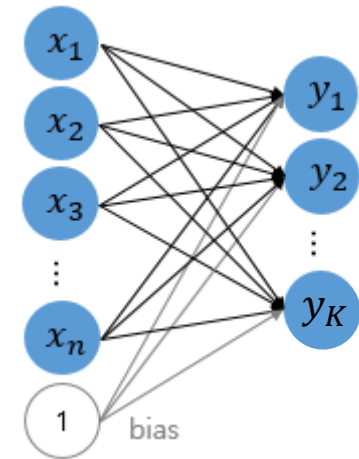
❖ 이진분류 =>
$$\epsilon_{CEE} = -\frac{1}{N} \sum_{n=0}^{N-1} \{y_n \ln p_n + (1 - y_n) \ln(1 - p_n)\}$$

$$\{y_n \ln p_n + (1 - y_n) \ln(1 - p_n)\}$$

$$= \{y_{0,n} \ln p_{0,n} + (y_{1,n}) \ln(p_{1,n})\}$$

$$= \sum_{k=0}^1 y_{k,n} \ln p_{k,n} \Rightarrow \sum_{k=0}^{K-1} y_{k,n} \ln(p_{k,n})$$

$$\epsilon_{CEE} = -\frac{1}{N} \sum_{n=0}^{N-1} \sum_{k=0}^{K-1} y_{i,n} \ln(p_{i,n})$$



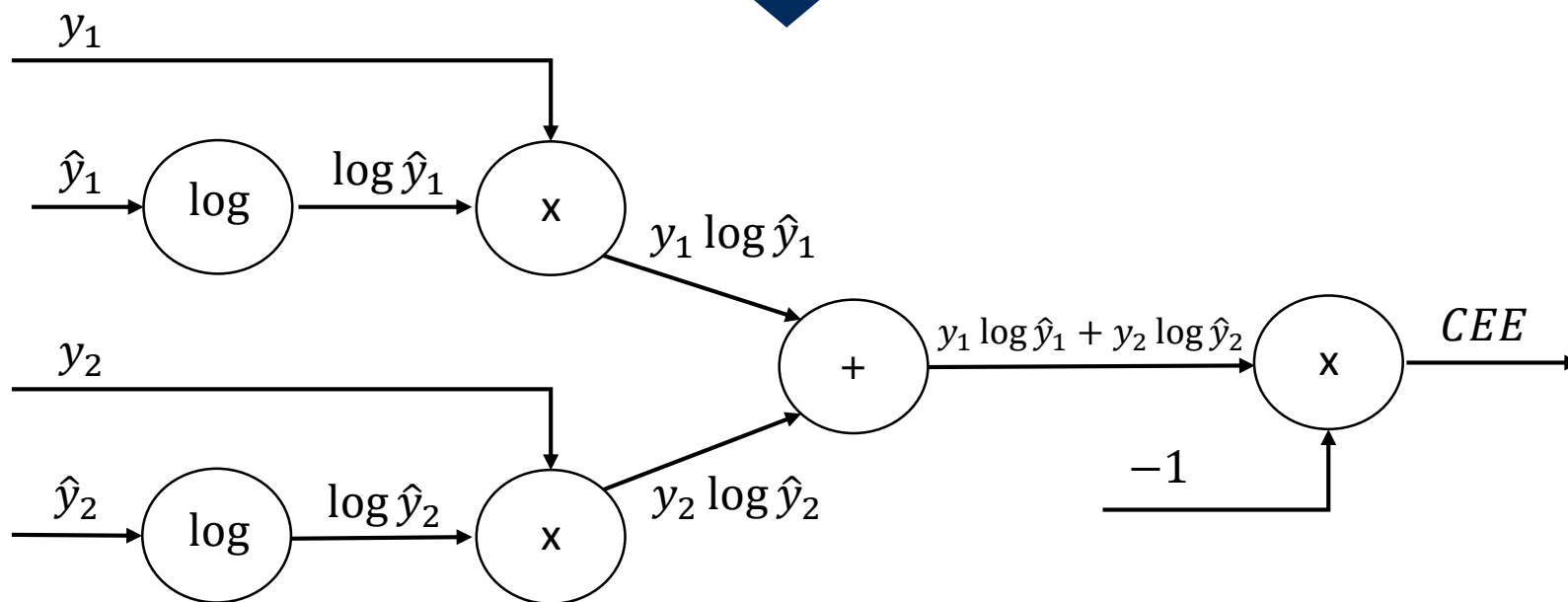
[참고] Cross Entropy Error (CEE)



CEE Forward

$$CEE = - \sum_k y_k \log \hat{y}_k$$

forward

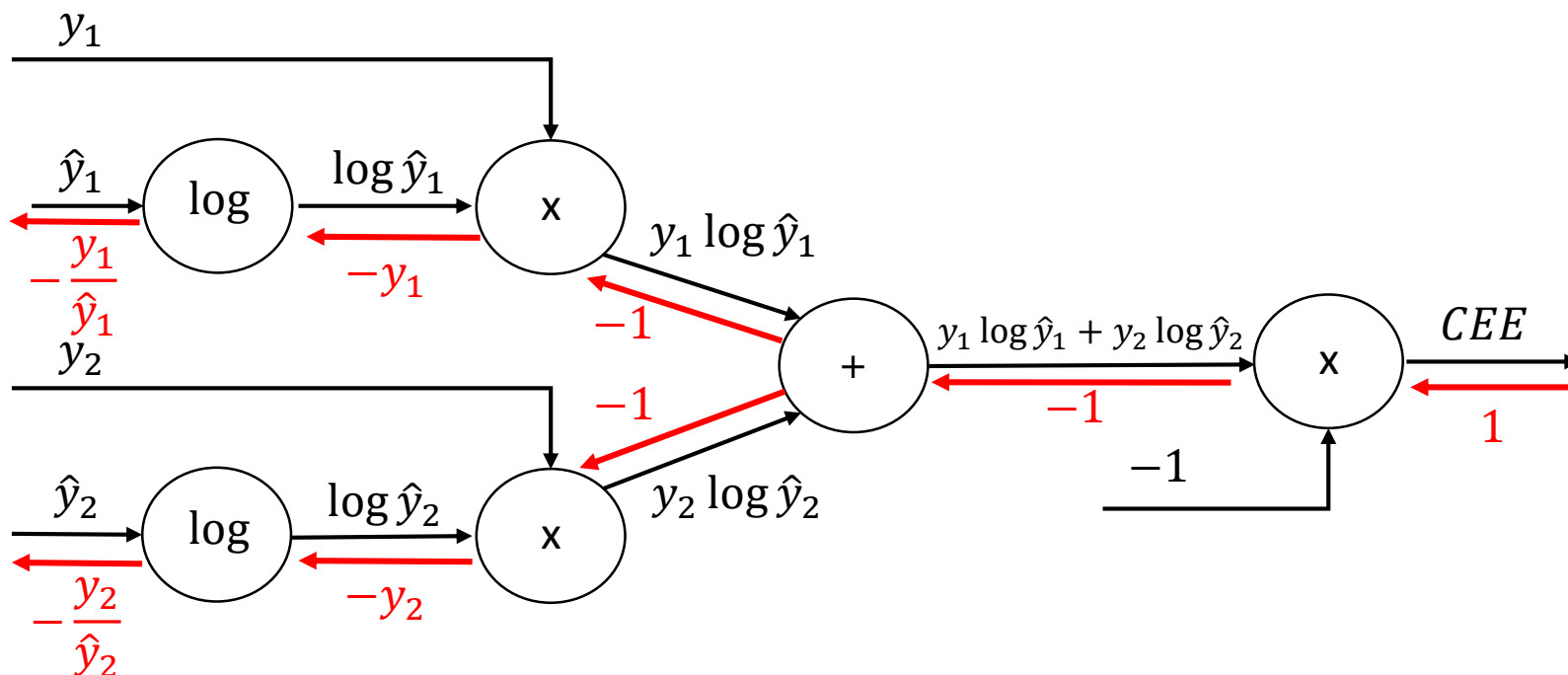


[참고] Cross Entropy Error (CEE)



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

CEE Backward



Chap.2 실습 #2



각 실습문제에 대해 code(+ 주석), 그래프, 분석 내용 등은 필수적으로 포함시킬 것

실습 #2) Softmax & CEE 실습

- Chap.1 실습 #2)에서 출력층을 제외한 모든 활성화 함수 자유
- Chap.1 실습 #2)에서 출력층 활성화 함수 Softmax로 변경
- Chap.1 실습 #2)에서 Loss function CEE로 변경
- Train set, Test set을 분할하여 사용
- 정확도 비교 이외에도 여러 분석 내용 기재

가상환경 생성



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

▪ Anaconda 가상환경 생성

관리자 권한 실행



자신만의
가상환경 생성



```
(base) C:\Windows\System32>conda create -n 2019140000 python=3.10 -y
```

↑
학번

가상환경 진입



```
(base) C:\Windows\System32>conda activate 2019140000  
(2019140000) C:\Windows\System32>_
```

↙ 가상환경에서 나갈 때는
conda deactivate

Tensorflow-gpu
설치



```
(2019140000) C:\Windows\System32>pip install tensorflow-gpu==2.10.0
```

numpy 설치



```
(2019140000) C:\Windows\System32>pip install numpy==1.26.4
```

▪ Anaconda 가상환경 생성

가상환경에
spyder 설치



```
(2019140000) C:WWindowsWSystem32>pip install spyder
```

Spyder 실행



```
(2019140000) C:WWindowsWSystem32>spyder
```

GPU 설정 확인



```
import tensorflow as tf  
print(tf.config.experimental.list_physical_devices('GPU'))
```

해당 출력이 나타난다면 성공

```
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

■ 모델 생성

- Dense : 완전연결층 / units : 사용할 노드 수 / activation : 활성화 함수

```
import tensorflow as tf
from tensorflow import keras
```

```
model = keras.models.Sequential([
    keras.layers.Dense(units = 1024, activation= 'sigmoid'),
    keras.layers.Dense(units = 1024, activation= 'sigmoid'),
    keras.layers.Dense(units = 1024, activation= 'sigmoid'),
    keras.layers.Dense(units = 512, activation= 'sigmoid'),
    keras.layers.Dense(units = 3, activation= 'sigmoid')
])
```

keras.layers.Flatten(), #입력을 그대로 flatten

입력이 1차원이 아닌 경우 Flatten 추가

■ Compile 함수

- 학습 방식에 대한 환경 설정을 하는 함수
 - Optimizer : 훈련 과정 ex) adam, sgd 등 -> 여기서 Learning Rate도 설정 가능
 - Loss Function : 손실함수 ex) mse, binary_crossentropy 등
 - Metrics : 평가지표 ex) accuracy (분류 시), mse(회귀 시) 등

```
optimizer = keras.optimizers.SGD(lr = 0.1)
model.compile(loss='categorical_crossentropy',
              optimizer = optimizer, metrics=["accuracy"])
```

- 머신러닝의 학습단위

- Epoch** : 사전적 의미 → "시대"

모든 훈련 데이터셋을 학습하는 횟수 ex) 10 epoch = 데이터셋 전체를 10번 반복

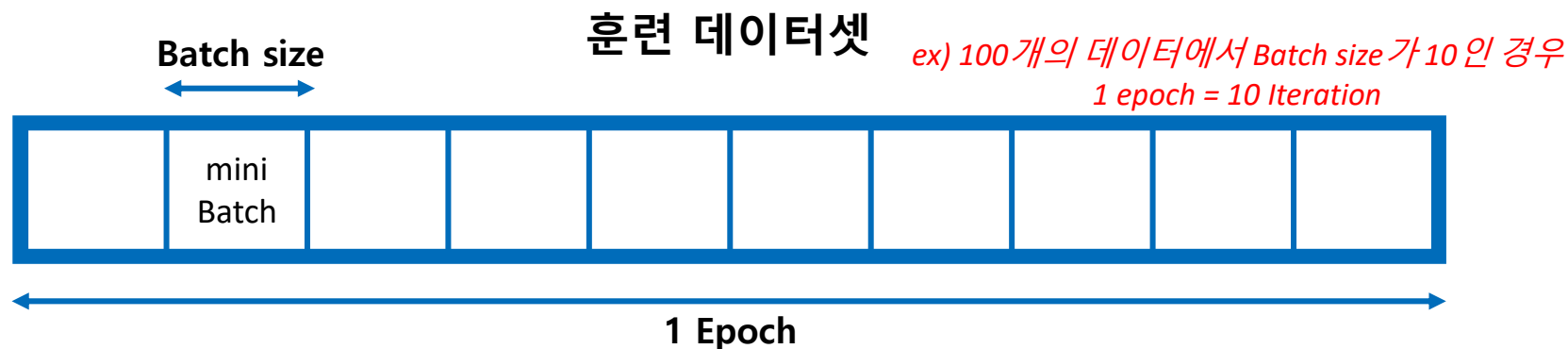
- Batch size** : 사전적 의미 → "집단", "한 회분", "함께 묶다"

Weight 업데이트 1회에 들어가는 데이터의 개수

1 Batch size에 해당하는 데이터셋 → mini Batch

- Iteration** : 사전적 의미 → "반복"

전체 훈련 데이터셋을 한번 학습시키는데 필요한 Batch의 수
즉, 1 epoch을 마치는데 필요한 Weight 업데이트 횟수



■ Fit 함수

- 모델 학습을 위한 함수

- Batch Size : Weight 업데이트 1회에 들어가는 데이터의 개수
 - ex) 값이 크면 클 수록 메모리를 많이 사용하는 대신, 속도가 빨라짐
 - 값이 작으면 학습은 꼼꼼하게 이루어질 수 있지만 학습 시간이 많이 걸림
- Epochs : 모든 훈련 데이터셋을 학습하는 횟수
 - ex) Epochs가 100이면 100번 반복
 - 여러 번 학습할 수록 효과는 커지지만 너무 많이 했을 경우 Overfitting 발생

```
history = model.fit(x_train, t_train, epochs = 100,  
                    validation_data = (x_test, t_test))
```

```
Epoch 98/100  
20/20 [=====] - 0s 9ms/step - loss: 0.2926 - accuracy: 0.8968 - val_loss: 0.3251 - val_accuracy: 0.8704  
Epoch 99/100  
20/20 [=====] - 0s 9ms/step - loss: 0.2889 - accuracy: 0.8841 - val_loss: 0.3253 - val_accuracy: 0.8815  
Epoch 100/100  
20/20 [=====] - 0s 9ms/step - loss: 0.2839 - accuracy: 0.8968 - val_loss: 0.3194 - val_accuracy: 0.8815
```

NN_data.csv를 사용

Keras를 사용하여 실습



■ CPU vs GPU

```
In [58]: from tensorflow.python.client import device_lib
...:
...: print(device_lib.list_local_devices())
```

연상에 사용할 수 있는 장치 확인 가능

```
# CPU 학습
print("CPU를 사용한 학습")
with tf.device("/device:CPU:0"):
    history = model.fit(x_train, t_train, epochs = 100,
                        validation_data = (x_test, t_test))

print("GPU를 사용한 학습")
with tf.device("/device:GPU:0"):
    history = model.fit(x_train, t_train, epochs = 100,
                        validation_data = (x_test, t_test))
```

```
Epoch 99/100
20/20 [=====] - 0s 9ms/step - loss: 0.9315 - accuracy: 0.5317 - val_loss: 0.8404 - val_accuracy: 0.7926
Epoch 100/100
20/20 [=====] - 0s 9ms/step - loss: 0.9888 - accuracy: 0.5460 - val_loss: 0.8607 - val_accuracy: 0.5704
```

CPU 사용

```
Epoch 99/100
20/20 [=====] - 0s 5ms/step - loss: 0.3105 - accuracy: 0.8889 - val_loss: 0.3071 - val_accuracy: 0.8889
Epoch 100/100
20/20 [=====] - 0s 5ms/step - loss: 0.3074 - accuracy: 0.8921 - val_loss: 0.3147 - val_accuracy: 0.8852
```

GPU 사용

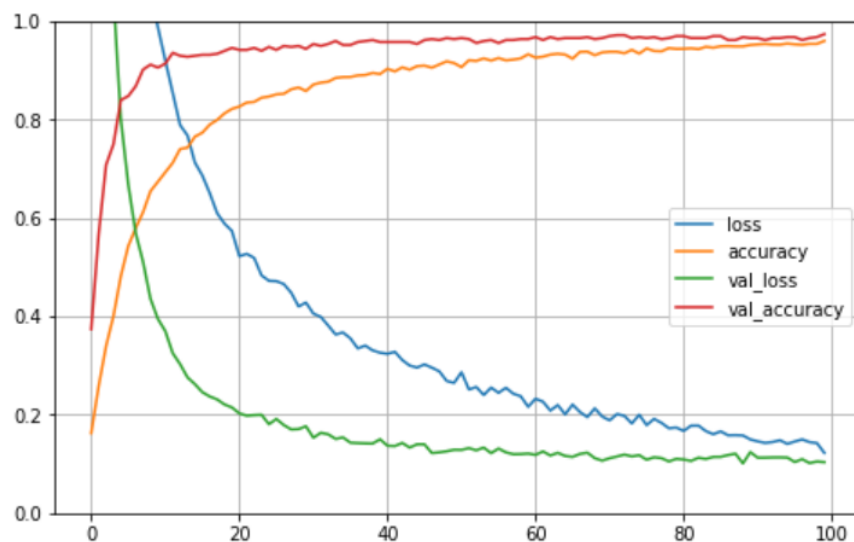
Keras를 사용하여 실습



■ 학습 결과 확인

```
import matplotlib.pyplot as plt
import pandas as pd

pd.DataFrame(history.history).plot()
plt.grid(True)
plt.gca().set_ylim(0.1)
plt.show()
```



Chap.2 실습 #3



각 실습문제에 대해 code(+ 주석), 그래프, 분석 내용 등은 필수적으로 포함시킬 것

실습 #3) Keras를 활용하여 Deep Neural Network 구현

- 실습 예시에서 사용한 Dense 층을 활용하여 DNN 구현
- Chap.1에서 사용한 "NN_data.csv" 데이터 사용
- label 데이터는 One-Hot Encoding 하여 사용
- Train set, Test set을 분할하여 사용
- GPU vs CPU 사용에 따른 훈련 시간 비교
- Activation Function: Sigmoid vs ReLU vs LeakyReLU 간 비교